

Intro to DevOps

Practice Questions

INSTRUCTIONS:

For the exam know/prepare the following:

- Quay.io account
- Red Hat Academy account
- Docker Hub account
- Infoeduka

LO1 — Use of containers and container services

1. Run an httpd container detached, publish container port 80 to host port 8081, give it a custom --name and a custom --hostname, and confirm both with podman inspect.
2. Run a container with --restart=always, kill its main process, and prove podman restarts it (check podman ps and the restart behavior).
3. Run a container with --memory=256m --cpus=0.5 and verify the limits are applied using podman stats and podman inspect.
4. Start one container with --env KEY=VALUE and another with --env-file, then show the environment differences with podman exec <c> env.
5. Create a user-defined bridge network with podman network create, run a container attached to it, and inspect the network to find the container's IP.
6. Pause and unpause a running container and explain what state its processes are in while paused.
7. Rename an existing container with podman rename and confirm the change.
8. Use podman logs with --tail, --since, and -f, and explain what each flag does.
9. Extract a single field (e.g. the IP address or restart count) using podman inspect --format '{{ ... }}' Go-template syntax.
10. Copy a file from the host into a running container and back out again with podman cp.
11. Exec an interactive shell into a running container, install a package, and explain whether that change survives the container being recreated.
12. Use podman top and podman stats to show the processes and live resource usage of a container.
13. Run a container with --rm, and explain when auto-removal is useful and what you lose by using it.
14. Bind-mount a host directory into a container read-only (-v hostdir:/path:ro) and prove that writes from inside are rejected.
15. Add custom --labels to two containers, then list only the ones matching a label using podman ps --filter.
16. Commit a modified running container into a new image with podman commit, then run a container from that image.
17. Show the filesystem changes made inside a running container with podman diff and interpret the A/C/D markers.
18. Run a container with --network none, demonstrate it has no external connectivity, and explain a use case.
19. Publish a port bound to a specific host IP only (-p 127.0.0.1:8082:80) and verify it is not reachable on the VM's other IPs.
20. Use podman port to list a container's published ports and cross-check with podman inspect.
21. Run a container as a non-root user with --user, and verify the effective UID inside with id / whoami.
22. Generate a systemd unit (or a Quadlet .container file) for a container and explain how it makes the container start on boot.
23. Run podman events in one terminal while you start/stop a container in another, and describe the lifecycle events you observe.

LO2 — Managing and creating container images

1. Write a Containerfile that uses an ARG to choose the base-image tag at build time (--build-arg), and build it twice with different values.
2. Write a multi-stage Containerfile that builds an artifact in one stage and copies only the result into a minimal final image; compare the final image size to a single-stage build.
3. Add a .containerignore file and demonstrate that ignored files are excluded from the build context / not copied.
4. Build an image and apply three tags at once (name:1.0, name:1, name:latest); explain the registry/namespace/name:tag convention.
5. Inspect an image's layer history with podman history and explain how to reduce the number of layers.
6. Write a Containerfile that sets a non-root USER and a writable WORKDIR; run it and confirm whoami is not root.
7. Add a HEALTHCHECK to a Containerfile, run the container, and show the health status in podman ps / via podman healthcheck run.
8. Demonstrate the difference between ENTRYPOINT and CMD in one Containerfile that uses both, then override CMD at podman run time.
9. Explain the difference between ADD and COPY and write a Containerfile that justifies using each.
10. Save an image to a tarball with podman save, remove it, then restore it with podman load; explain when you'd do this (air-gapped transfer).
11. Pull an image by digest (name@sha256:...) and explain why it's more reproducible than a tag.
12. Build an image with --no-cache and explain layer caching and when disabling it helps or hurts.
13. Build from a non-default Containerfile name using -f, and explain the build-context argument.
14. Use COPY --chown to set file ownership in the image and verify with ls -l inside the container.
15. Write a Containerfile combining ARG and ENV (build-time vs run-time variables) and demonstrate the difference.
16. podman login to a registry and push an image to a private repository; explain where/how the credentials are stored.
17. Remove dangling images with podman image prune and a specific image with podman rmi; explain what "dangling" means.
18. Write a Containerfile for a static file server using python -m http.server over a directory you COPY in, expose the port, and run it.
19. Build an image that installs a specific pinned version of a package (apt or apk) and explain why pinning matters for reproducibility.
20. Inspect an unknown image with podman image inspect to discover its default Entrypoint/Cmd, exposed ports, and env, then run it accordingly.
21. Write a Containerfile with a multi-line RUN ... && ... that installs packages and cleans the package cache in the same layer; explain why cleanup must share the layer.
22. Tag and push the same image to two different registries (e.g. Docker Hub and Quay) and explain when image mirroring is useful.
23. Run an image and list the OS packages installed inside it; explain the security/size value of choosing a minimal base image.

LO3 —Application delivery using containers, network architecture, and component security

1. Create a pod with `podman pod create` (publishing a port), add an app container and a database container to it, and show they reach each other over localhost (shared network namespace).
2. Create a user-defined network, run an app container and a postgres container on it, and prove the app resolves the database by container name (podman DNS on custom networks).
3. Deploy a two-tier stack of your choice that is not Drupal/Joomla/WordPress (e.g. Ghost + MySQL, Gitea + PostgreSQL, or Redmine + PostgreSQL) and complete its first-run setup.
4. Persist the database's data in a named volume so it survives `podman rm` + re-creation of the DB container; prove the data is still there afterward.
5. Use a podman secret (`podman secret create + --secret`) to pass the database password instead of `--env`; explain the security benefit.
6. Write a podman compose file (`compose.yaml`) for an app + database, bring it up with `podman compose up -d`, and show both services running.
7. In that compose file, keep the database on an internal network with no published port and expose only the app; prove the DB isn't reachable from the host.
8. Add a compose healthcheck plus `depends_on`: `condition: service_healthy` so the app waits for the database; demonstrate the startup ordering.
9. Use an `env_file` in compose for the database credentials instead of inline values.
10. Scale one compose service to multiple replicas (`podman compose up --scale app=3`) and explain how requests are distributed.
11. Mount a bind mount for app configuration and a named volume for data in the same container, and explain why you'd use each.
12. Back up a named volume to a tarball using a helper alpine/busybox container that mounts the volume, then restore it into a fresh volume.
13. Put an nginx reverse-proxy container in front of an app container on a shared network and route host traffic through nginx to the app.
14. Add a database-admin container (e.g. adminer or pgadmin) to the network and use it to inspect the database.
15. Generate Kubernetes YAML from a running pod with `podman generate kube` and explain how it bridges LO3 to LO4.
16. Run a pod from a Kubernetes YAML with `podman kube play`, then tear it down with `podman kube play --down`.
17. Set per-service CPU/memory limits in a compose file and verify them with `podman stats`.
18. Inspect a pod with `podman pod inspect` and show which Linux namespaces the containers share (network, IPC) and which they don't (PID by default).
19. Demonstrate name-based discovery failing when two containers are on different networks, then fix it by attaching them to a common network.
20. Attach a single container to two networks (a frontend and a backend) and explain the segmentation benefit.
21. Use `podman compose logs` and `podman compose ps` to observe and troubleshoot a multi-service stack.
22. Share configuration across two replicas of the same app via a shared named volume, and explain a risk of shared read-write storage.
23. Tear down a stack with `podman compose down`, then explain what happens to named volumes and how `--volumes` changes that.

LO4 — Accelerated delivery of multilayer applications using containers.

1. Create a Deployment named web running nginx:1.25 with 3 replicas using a single imperative kubectl create deployment command, then export its generated YAML to a file.
2. **Enable pulling images from DockerHub as an authenticated user to lower the chance of hitting the pull limit. What kind of resource do you need to create?**
3. Scale web from 3 to 5 replicas two different ways — once with kubectl scale, once by editing the manifest — and show the ReplicaSet that results.
4. Perform a rolling update of web from nginx:1.25 to nginx:1.27 and watch it with kubectl rollout status.
5. View a Deployment's rollout history and roll back to the previous revision. Explain what the CHANGE-CAUSE column shows and how to populate it.
6. Set the update strategy to RollingUpdate with maxSurge: 1 and maxUnavailable: 0, and explain the effect on availability during an update.
7. Switch a Deployment's strategy to Recreate and describe a concrete scenario where this is required instead of RollingUpdate.
8. Add CPU/memory requests and limits to a Deployment's container and verify they appear in the running pod spec.
9. Set revisionHistoryLimit: 3 and explain how it affects your ability to roll back and how many old ReplicaSets are kept.
10. Set a Deployment's image to a non-existent tag, observe the rollout get stuck, and explain why the old pods keep serving traffic.
11. Use a label selector to list only the pods belonging to one Deployment, and explain the Deployment → ReplicaSet → Pod selector relationship.
12. Expose a Deployment with kubectl expose, then explain what object was created and how its selector was derived.
13. Add a sidecar (second) container to a Deployment's pod template and explain how the two containers share the pod network and volumes.
14. Write a Deployment manifest for httpd:2.4 with 2 replicas, a named container port, and a nodeSelector; explain why it stays Pending if no node matches.
15. Create a bare Pod (no controller) running busybox that sleeps, then explain what happens when you delete it versus a Deployment-managed pod.
16. Write a StatefulSet for redis:7 with 3 replicas plus a headless Service, and show the stable pod names and per-pod DNS records.
17. Create a DaemonSet running a busybox agent and explain why exactly one pod runs per node.
18. Create a Job using busybox/perl that computes something once and completes; show COMPLETIONS and read the result from logs.
19. Create a CronJob that prints the date every minute; show how to suspend it and how to list the Jobs it spawns.
20. Manually run a Job from an existing CronJob (kubectl create job --from=cronjob/...) to test it on demand.
21. Show that a StatefulSet creates and terminates pods in order, and contrast this with a Deployment.
22. Create a multi-container Pod where two containers share an emptyDir — one writes a file, the other reads it. Prove it's shared.
23. List the StorageClasses, PVs and PVCs.
24. Mount a ConfigMap as a volume so each key becomes a file, and verify the contents inside the pod.
25. Mount a single ConfigMap key to a specific path using subPath; explain when it's needed and its update caveat.
26. Mount a Secret as a volume and verify the files contain decoded values with restrictive permissions.
27. Show that scaling a StatefulSet creates one PVC per replica, then explain what happens to those PVCs when the StatefulSet is deleted.
28. Use an initContainer to pre-populate data into a shared volume before the main container reads it.
29. Set readOnly: true on a volume mount and prove writes are rejected.
30. Set a sizeLimit on an emptyDir and explain what happens if the container exceeds it.
31. Create a generic Secret from literals for a DB username/password, then show the stored values are base64-encoded, not encrypted.
32. Create a Secret from files (--from-file) holding a certificate and key, and identify the resulting keys.

33. Create a kubernetes.io/tls typed Secret from a cert/key pair and explain where it's consumed.
34. Mount a Secret as a volume and explain the security trade-offs versus env vars.
35. Use envFrom with a secretRef to load every key of a Secret as env vars at once.
36. Create a docker-registry (image-pull) Secret and reference it via imagePullSecrets; explain when it's required.
37. Create a Secret with several keys and selectively mount only one of them into a pod.
38. Create a ClusterIP Service for a Deployment and resolve it by DNS from another pod (nslookup <svc>.<ns>.svc.cluster.local).
39. Create a NodePort Service and reach it via minikube service <svc> --url and \$(minikube ip):<nodePort>.
40. Create a LoadBalancer Service, explain why EXTERNAL-IP is <pending> on minikube, and obtain access with minikube tunnel.
41. Create a headless Service (clusterIP: None) for a StatefulSet and show it returns per-pod A records instead of a single VIP.
42. Inspect a Service's Endpoints/EndpointSlice and explain how they're populated from the pod selector.
43. Demonstrate cross-namespace access using the FQDN, and show the short name fails from another namespace.
44. Compare kubectl port-forward to a Service versus to a Pod — explain the difference and when each fits.
45. Explain the difference between a Service's port, targetPort, and nodePort.
46. Verify connectivity to a ClusterIP Service from a throwaway debug pod (kubectl run tmp --rm -it --image=busybox -- sh) with wget/nc.
47. Create two Deployments and one Service that load-balances across both by sharing a common label; prove requests hit both.
48. Systematically diagnose why a pod can't reach a Service: DNS → endpoints → selector labels → readiness → port.
49. Expose a service with a NodePort and verify it works.
50. Add an HTTP livenessProbe hitting / on port 80 to an nginx Deployment and verify via kubectl describe pod.
51. Add a tcpSocket readiness probe to a redis container on port 6379 and verify it.
52. Configure a startup probe with a failureThreshold * periodSeconds budget for a ~2-minute boot and justify the numbers.
53. Explain the default behavior when no probes are defined at all.

LO5 — Solve problems with application shipping by using containers

For each command: identify the mistake, fix it, and explain what was wrong.

1. `podman run -d --name web -p 80:8080 docker.io/library/nginx`
(nginx listens on 80; the site isn't reachable on the host port you expected — what's reversed?)
2. `podman run -d --name db -e MYSQL_ROOT_PASSWORD docker.io/library/mysql`
(the container exits during init — inspect podman logs.)
3. `podman run -d --name app --network host -p 8080:80 docker.io/library/nginx`
(why is the `-p` flag effectively ignored here?)
4. `podman run -d --name c1 docker.io/library/busybox`
(the container goes straight to Exited (0) — why, and how do you keep it running?)
5. `podman run --rm -d --name job docker.io/library/alpine echo hello`
(then podman logs job fails — explain the `--rm` + detached gotcha.)
6. `podman run -d -p 8080:80 --memory 8m docker.io/library/mysql`
(the database never becomes healthy — what limit is the problem?)
7. `podman run -d --name a docker.io/library/alpine sleep 1d`
`podman run -d --name b docker.io/library/alpine sleep 1d`
`podman exec a ping b`
(name resolution fails — why, on the default network, and how do you fix it?)
8. `podman run -d --name web -v ./html:/usr/share/nginx/html docker.io/library/nginx`
(the files aren't visible in the container, or SELinux denies access — what mount flag is missing?)
9. For each Containerfile/Dockerfile: identify the mistake, fix it, and explain what was wrong.
10. `FROM debian:12`
`RUN apt-get install -y nginx`
(the build fails to find the package — what's missing before install?)
11. `FROM python:3.11`
`COPY app.py /app/app.py`
`WORKDIR /app`
`CMD python app.py`
(the app starts but doesn't handle signals / can't be stopped cleanly — shell vs exec form?)
12. `FROM node:20`
`COPY . /app`
`WORKDIR /app`
`RUN npm install`
(every source change triggers a full npm install — reorder for layer caching.)
13. `FROM alpine:3.20`
`ENV PATH=/app/bin`
`RUN apk add --no-cache curl`
(after this, `curl/sh` aren't found — what did setting `PATH` break?)
14. `FROM ubuntu:24.04`
`EXPOSE 8080`
`CMD ["python3", "-m", "http.server", "3000"]`

15. (you map -p 8080:8080 but nothing answers — what mismatch is there, and what does EXPOSE actually do?)
16. FROM debian:12
RUN apt-get update && apt-get install -y build-essential
(the image is huge — how do you cut the size in the same layer?)
17. FROM python:3.11
USER appuser
COPY requirements.txt /app/requirements.txt
RUN pip install -r /app/requirements.txt
(permission denied errors — what's wrong with the USER/COPY ordering and ownership?)
18. FROM golang:1.22
COPY . /src
WORKDIR /src
RUN go build -o app .
CMD ["/src/app"]
(the final image is ~1 GB — how would a multi-stage build fix it?)
19. A pod is stuck Pending. Walk through the diagnosis with kubectl describe pod and identify the reason from the events (scheduling / resources / PVC / nodeSelector).
20. Find a deployment from older tasks, remove a couple of spaces and try to deploy it. Identify the cause in the error messages and fix it.
21. A pod is in ImagePullBackOff. Diagnose the cause (image typo, missing tag, private registry without pull secret) and fix it.
22. A pod is in CrashLoopBackOff. Use kubectl logs --previous to read the last crash and fix the root cause.
23. A pod's last state shows OOMKilled. Confirm it from kubectl get pod -o yaml / describe, then fix it (raise the memory limit or fix the app).
24. A Deployment's pods never become Ready. Trace it to a failing readiness probe and correct the probe or the app.
25. A Service returns nothing. Diagnose a Service/pod label mismatch using kubectl get endpoints.
26. Given a manifest where spec.selector.matchLabels doesn't match spec.template.metadata.labels, explain the error kubectl apply returns and fix it.
27. Given a manifest whose resources.requests exceed any node's capacity, explain why the pod is Pending (Insufficient cpu/memory) and fix it.
28. Given a pod mounting a PVC that doesn't exist, diagnose the FailedMount/Pending and create the PVC.
29. A container based on ubuntu with no long-running command shows Completed/CrashLoopBackOff. Explain why and add a proper command.
30. Use kubectl get events --sort-by=.lastTimestamp (or kubectl events) to triage everything that happened to a pod over time.
31. Use kubectl exec -it to get a shell in a pod and debug DNS with nslookup and cat /etc/resolv.conf.
32. Use an ephemeral debug container (kubectl debug -it <pod> --image=busybox --target=<container>) to troubleshoot a no-shell/distroless container.
33. Spin up a throwaway kubectl run tmp --rm -it --image=busybox pod to test connectivity to a Service from inside the cluster.
34. A node shows NotReady. List what you'd check (kubelet, disk pressure, CNI) and inspect it on minikube with kubectl describe node and sudo minikube logs.
35. A pod is stuck Terminating. Explain finalizers and the grace period, then force-delete it (--grace-period=0 --force) and state the risks.
36. Given a manifest with the wrong apiVersion/kind pairing (e.g. apps/v1 + Pod), explain the validation error and correct the group/version.
37. A pod fails with CreateContainerConfigError because a configMapKeyRef key is missing. Diagnose and fix.
38. A pod can't mount a Secret that lives in a different namespace. Explain namespace scoping and fix it.

39. A rollout is stuck with ProgressDeadlineExceeded. Use `kubectl describe deployment / rollout status` to find the cause and remediate.
40. Catch indentation/field typos (e.g. `imagePullpolicy`, misnested ports) before applying with `kubectl apply --dry-run=server / --validate=true`, then fix them.
41. An app's logs say it can't reach its database Service. Verify in order: pod running → Service exists → endpoints populated → DNS resolves → correct port — and identify the broken link.
42. Read a container's `state/lastState` and `restartCount` with `kubectl get pod -o jsonpath` to determine the root cause of repeated restarts.
43. Compare OpenShift Route to Ingress.

LO6 — Evaluate the use of selected container orchestration systems - theoretical

1. Analyze the networking model of Kubernetes versus Docker's network.
2. Evaluate Kubernetes storage abstraction (CSI, StorageClasses, dynamic provisioning) against Podman's storage plugins. Which is more flexible for stateful workloads, and why?
3. Evaluate the operator pattern (CRDs + controllers) for managing stateful services versus using only k8s manifests, in terms of control and effort.
4. Assess the developer experience of plain Kubernetes (kubectl + manifests) versus OpenShift's Source-to-Image (S2I) and developer console. What does each optimize for?
5. Critically evaluate migrating a workload from OpenShift to Kubernetes.
6. Evaluate running CI/CD build agents (Jenkins agents, GitLab runners, Tekton) inside Kubernetes versus on dedicated VMs, considering isolation, autoscaling, and noisy-neighbor effects.
7. How does Kubernetes integrate with cloud environments and dynamic capacity provisioning?
8. Critically evaluate the default security and hardening of OpenShift versus vanilla Kubernetes. Why do enterprises in regulated industries often choose OpenShift?
9. Critically evaluate the learning curve and required skill set of OpenShift versus plain Kubernetes. Does OpenShift's added abstraction help or hinder a team new to containers?
10. Compare the resource footprint of full Kubernetes versus k3s for a small on-premise deployment. When is full Kubernetes overkill?
11. Defend whether a small team should use Docker Compose on a single host or move straight to Kubernetes, considering complexity, scaling needs, and resilience.
12. Analyze vendor lock-in across Kubernetes (open source / CNCF) and OpenShift (Red Hat). How does the choice affect long-term flexibility?
13. Defend a recommendation of OpenShift over vanilla Kubernetes for a company that wants an integrated, vendor-supported platform with built-in developer and operations tooling.
14. Compare storage provisioning between Kubernetes and regular VM workloads.
15. A startup with two engineers must ship a containerized product quickly and cheaply. Recommend container solution and justify your choice on cost and operational overhead.
16. Compare Docker and Podman in terms of architecture (daemon vs daemonless) and rootless security. Why might a security-conscious team prefer Podman?
17. Compare the day-2 operational overhead (cluster upgrades, scaling nodes, backups) of self-managed Kubernetes versus a managed Kubernetes service.
18. A media-streaming company expects spiky, unpredictable global traffic. Recommend a containerized solution. Elaborate your choice.
19. Defend whether a company that has outgrown Docker Swarm (or a single Compose host) should migrate to Kubernetes — weigh the migration effort against the long-term benefits.
20. Would you choose Kubernetes as a solution for a microservices architecture, focus on its orchestration, resilience and ecosystem. Elaborate your standing with arguments.
21. Would you choose Kubernetes for enterprise-grade applications, focus on its scalability, community support and feature richness. Elaborate your standing with arguments.
22. Would you choose OpenShift as a solution for a microservices architecture, focus on its orchestration, resilience and ecosystem. Elaborate your standing with arguments.
23. Would you choose OpenShift for enterprise-grade applications, focus on its scalability, community support and feature richness. Elaborate your standing with arguments.